

UNIVERSIDADE ESTADUAL DE SANTA CRUZ — UESC

Escape Room Cooperativo:
Documentação da Aplicação e do Protocolo ERP/1.0

Projeto da disciplina de Redes de Computadores

Alunos: Arthur Araújo, Bruno Cardoso, João Pedro França e Lucas Vieira

Ilhéus - BA
2026

Descrição

Este documento descreve a aplicação Escape Room Cooperativo e o protocolo de aplicação ERP/1.0 (Escape Room Protocol, versão 1.0). O sistema consiste em um jogo cooperativo para 2 a 4 jogadores, executado em terminal, em arquitetura cliente-servidor sobre sockets TCP.

A documentação cobre o propósito da aplicação, a arquitetura geral, a escolha do protocolo de transporte, os requisitos mínimos, o funcionamento da mecânica de jogo, as facilidades de digitação para o usuário, o roteamento de mensagens unicast e broadcast, o chat, o subprotocolo de descoberta automática de servidor por UDP broadcast e as limitações conhecidas.

Sumário

1. Propósito da aplicação
2. Arquitetura geral da solução
3. Motivação da escolha do protocolo de transporte
4. Requisitos mínimos de funcionamento
5. Funcionamento da aplicação
6. Protocolo de aplicação ERP/1.0
7. Subprotocolo de descoberta de servidor (UDP broadcast)
8. Limitações conhecidas
9. Instruções de execução

1. Propósito da aplicação

A aplicação implementa um jogo de Escape Room cooperativo em rede local. Entre 2 e 4 jogadores entram na mesma partida, são distribuídos entre dois caminhos ou papéis diferentes do mapa e precisam resolver enigmas que dependem um do outro, trocando informações pelo chat embutido no jogo, até alcançarem juntos a saída dentro de um tempo limite.

O objetivo didático do projeto é exercitar, na prática, conceitos centrais da disciplina de Redes de Computadores: comunicação cliente-servidor sobre sockets, definição de um protocolo de aplicação próprio, delimitação de mensagens, tratamento de erros, concorrência com múltiplos clientes e gerenciamento de estado global compartilhado.

O servidor é a única fonte de verdade do jogo: ele mantém o estado de cada jogador, das salas, dos objetos, dos enigmas, do tempo restante e da condição de vitória ou derrota. Os clientes nunca alteram o estado diretamente; eles apenas enviam comandos e recebem atualizações processadas e autorizadas pelo servidor.

2. Arquitetura geral da solução

A solução é dividida em módulos independentes, mas integrados:

Arquivo	Responsabilidade
server.py	Servidor TCP. Aceita conexões, mantém a máquina de estados da partida, processa mensagens e decide se a resposta será enviada a um cliente específico, a todos ou apenas aos jogadores de uma sala.
client.py	Cliente de terminal. Envia comandos digitados pelo jogador, executa o handshake de entrada, renderiza mensagens do servidor e preserva o prompt durante mensagens assíncronas.
launcher.py	Interface de conveniência. Permite criar uma partida local ou encontrar automaticamente um servidor na rede por UDP broadcast.
game/protocol.py	Define tipos de mensagens, constantes do jogo, estados do servidor, códigos de erro e funções de serialização JSON.
game/state.py	Contém a lógica de jogo: jogadores, inventário, salas, enigmas, parser de comandos, aliases, movimentação, anti-softlock e atualizações de sala.
game/rooms.py	Define o mapa Hospital Abandonado, seus caminhos, salas, objetos, saídas, senhas, dicas e papéis necessários.

O servidor é single-process e multi-thread: a thread principal aceita conexões e cada cliente é atendido por uma thread dedicada. Como várias threads podem tentar ler ou alterar a mesma instância de GameState e o mesmo dicionário de conexões, o servidor protege as seções críticas com threading.Lock.

Assim, ações concorrentes, como dois jogadores tentando pegar o mesmo item, são serializadas e não corrompem o estado compartilhado.

3. Motivação da escolha do protocolo de transporte

O protocolo principal do jogo, ERP/1.0, roda sobre TCP. A escolha foi feita porque o jogo depende de uma máquina de estados compartilhada e sensível à ordem dos eventos.

- **Entrega confiável:** uma mensagem ACTION, GAME_OVER ou ROOM_UPDATE perdida poderia deixar clientes e servidor com percepções diferentes do jogo.
- **Ordem de chegada:** comandos como pegar chave ou usar chave só fazem sentido se as ações anteriores foram processadas na sequência correta.
- **Conexão persistente:** cada jogador mantém um socket TCP aberto durante a partida, o que simplifica a identificação da sessão e o envio de respostas.
- **Delimitação simples:** cada mensagem ERP/1.0 é um JSON seguido de quebra de linha, e o TCP preserva a ordem dos bytes para reconstrução correta do fluxo.

O UDP foi reservado apenas para uma função auxiliar: descoberta automática de servidor em rede local. Nessa etapa, a perda de um pacote não compromete o jogo porque o anúncio é reenviado periodicamente.

4. Requisitos mínimos de funcionamento

- Python 3.10 ou superior, pois o código usa anotações de tipo como `str | None`.
- Rede TCP/IP entre as máquinas. Em uma única máquina, pode ser usado 127.0.0.1. Em rede local, a porta TCP 5000 precisa estar acessível no host.
- Mesma sub-rede local apenas para a descoberta automática do launcher.py, pois UDP broadcast não atravessa roteadores por padrão.
- Porta UDP 5001 liberada se for usada a busca automática de servidor pelo launcher.py.
- Terminal com suporte a UTF-8 e sequências ANSI, já que o cliente usa acentuação, caracteres visuais e limpeza de linha de prompt.
- Módulo readline ou equivalente. Em Linux e macOS, geralmente já existe. No Windows nativo, pode ser necessário instalar pyreadline3 com `python -m pip install pyreadline3`, ou executar em WSL.
- Entre 2 e 4 jogadores por partida, conforme MIN_PLAYERS e MAX_PLAYERS.

5. Funcionamento da aplicação

5.1. Papéis, mapa e cooperação

O mapa atual, Hospital Abandonado, possui dois caminhos essenciais: role 0, associado ao caminho da Recepção, e role 1, associado ao caminho da Sala de Força. Os caminhos são inicialmente separados e convergem no Corredor Central.

A distribuição de papéis é alternada conforme a quantidade de jogadores conectados:

- 2 jogadores: role 0, role 1
- 3 jogadores: role 0, role 1, role 0
- 4 jogadores: role 0, role 1, role 0, role 1

Cada caminho contém pistas ou ações que ajudam o outro caminho. Por exemplo, uma informação encontrada em uma sala pode ser a senha necessária para outro jogador avançar. O jogo não anuncia automaticamente todo item encontrado; por isso, o comando chat <mensagem> é parte essencial da cooperação.

Alguns eventos cooperativos mais críticos, como energia religada, porta destrancada remotamente e reencontro no corredor central, são anunciados automaticamente via `PLAYER_EVENT` para evitar que uma virada de jogo importante passe despercebida.

5.1.1. Descrição das salas

Sala	Descrição
Recepção	Sala inicial dos jogadores do papel (role) 0. Contém objetos como o terminal de monitoramento e uma porta protegida por senha que permite avançar para novas áreas do mapa.
Sala de Força	Sala inicial dos jogadores do papel (role) 1. Possui o painel de controle responsável por restaurar a energia do hospital e uma porta protegida por senha.
Corredor Central	Área onde os dois caminhos do mapa convergem. Possui dispositivos que exigem cooperação entre os jogadores e permite o reencontro das equipes.
Ala Médica	Contém o cofre médico e pistas utilizadas para obtenção de itens importantes para a progressão da partida.
Subsolo	Área avançada do mapa que reúne desafios envolvendo a caixa de ferramentas, o servidor de TI e outros elementos necessários para a conclusão do jogo.
Saída	Destino final da partida. A vitória ocorre quando todas as condições necessárias para a abertura da saída forem satisfeitas dentro do tempo limite.

5.2. Comandos do jogador

Categoria	Comandos
Comandos gerais	ready; sair; chat <mensagem>; votar hospital; olhar; sala; inventario; dica
Ações de exploração	examinar <objeto>; pegar <objeto>
Ações de uso	usar <item> em <objeto>; colocar <senha> no <objeto>
Movimentação	ir norte; ir sul; ir leste; ir oeste

5.2.1. Normalização e facilitação da digitação

Para melhorar a usabilidade em uma interface de terminal, o jogo não exige que o jogador digite sempre o identificador interno exato dos objetos. Antes de interpretar a entrada, o motor de jogo aplica uma etapa de normalização: converte o texto para minúsculas, remove acentos, trata underscores e hífen como espaços e reduz múltiplos espaços para um único espaço.

Com isso, entradas como Inventário, inventario e INVENTARIO são equivalentes. Também é possível digitar nomes internos com espaços, sem underline. Por exemplo, dispositivo esquerdo pode ser interpretado como dispositivo_esquerdo, e placa de alimentacao pode ser interpretada como placa_de_alimentacao.

Além da normalização, o jogo possui aliases específicos para alguns objetos importantes. Esses aliases foram verificados no código e só funcionam quando o jogador está na sala correta e possui o papel que consegue ver aquele objeto.

Entrada aceita pelo jogador	Objeto interno resolvido	Condição de funcionamento
examinar terminal	terminal_de_monitoramento	Recepção, jogador do role 0
examinar painel / colocar 440 no painel	painel_de_controle	Sala de Força, jogador do role 1
colocar 8520 na porta	porta_leste	Recepção, porta interativa visível
colocar 1968 na porta	porta_sul	Sala de Força, porta interativa visível
examinar cofre / colocar 9999 no cofre	cofre_medico	Ala Médica
examinar caixa / colocar 1234 na caixa	caixa_de_ferramentas	Subsolo
examinar servidor / colocar 7701 no servidor	servidor_de_ti	Subsolo
usar chave direita em dispositivo direito	chave_direita + dispositivo_direito	Corredor Central, item no inventário

A resolução contextual de porta é uma regra específica: quando a sala possui uma única porta interativa visível, o jogador pode digitar colocar <senha> na porta, sem precisar escrever porta_leste ou porta_sul. Na Recepção, porta é resolvida como porta_leste; na Sala de Força, porta é resolvida como porta_sul.

Nem toda abreviação genérica é aceita automaticamente. Por exemplo, se não houver alias para uma palavra isolada, o jogador deve digitar um nome suficientemente próximo do objeto interno normalizado. Essa restrição evita ambiguidade quando existem vários objetos parecidos na mesma sala.

5.2.2. Sistema de dicas (dica)

Cada sala do mapa possui uma lista própria de dicas, definida estaticamente em game/rooms.py. O comando dica não revela todas as dicas de uma vez: a cada chamada, o servidor entrega a próxima dica da lista da sala em que o jogador se encontra, avançando um índice interno mantido por sala.

Esse índice é compartilhado entre os jogadores que percorrem o mesmo caminho, e não é reiniciado por jogador: se um jogador pedir uma dica e, em seguida, outro jogador da mesma sala pedir novamente, ele recebe a dica seguinte da lista, não a primeira. Quando todas as dicas de uma sala já foram entregues, novas chamadas de dica retornam a mensagem “Sem mais dicas.”.

O índice de dicas é reiniciado junto com o restante do estado do mapa sempre que a partida é reiniciada (vitória, derrota, perda de papel essencial ou cancelamento de contagem regressiva).

A resposta ao comando dica é sempre enviada via HINT, em unicast, apenas ao jogador que solicitou.

5.2.3. Preservação do prompt e conforto de digitação no cliente

O cliente também possui uma melhoria de interface para não atrapalhar o jogador enquanto ele digita. Como mensagens do servidor podem chegar a qualquer momento, o cliente usa uma função de impressão controlada que limpa a linha atual, imprime a mensagem recebida e redesenha o prompt junto com o texto que o usuário estava digitando.

Essa estratégia evita que mensagens assíncronas, como CHAT_BROADCAST, TIMER_UPDATE ou PLAYER_EVENT, quebrem visualmente o comando em construção. Em ambientes com readline disponível, o cliente consegue ler o conteúdo atual da linha de entrada para restaurá-lo após a impressão da mensagem.

5.3. Escolha de nome do jogador

Ao abrir o cliente, o jogador informa um nome de usuário. Se esse nome já estiver em uso por outro jogador conectado, o servidor responde ERROR [NAME_TAKEN]. O cliente de referência não libera o loop normal de comandos nesse momento; ele solicita outro nome ao usuário e envia novo JOIN na mesma conexão. O jogador só entra definitivamente no jogo depois que o servidor responde WELCOME.

5.4. Robustez contra desconexões e anti-softlock

Como o mapa depende de ambos os papéis essenciais, o servidor trata desconexões de forma diferente conforme o momento da partida.

- **No lobby:** se alguém sai, os papéis dos jogadores restantes são redistribuídos e o estado de ready é limpo.
- **Durante o COUNTDOWN:** se o número de jogadores cair abaixo do mínimo ou um papel essencial ficar descoberto, a contagem é cancelada e todos voltam ao lobby.
- **Durante a partida:** se restarem menos de 2 jogadores, o jogo termina em derrota.
- Durante a partida, se um papel essencial ficar vazio, a partida é reiniciada automaticamente: mapa resetado, inventários limpos, papéis redistribuídos e jogadores de volta ao lobby.
- Se a partida puder continuar, os itens do inventário do jogador desconectado são devolvidos ao chão da sala onde ele estava, permitindo que outro jogador do mesmo caminho recupere esses itens sem refazer puzzles já resolvidos.

Essa lógica evita dois tipos de softlock: começar ou continuar sem um caminho essencial ativo, ou perder para sempre um item essencial porque o jogador que o carregava desconectou.

6. Protocolo de aplicação ERP/1.0

6.1. Formato geral das mensagens

Toda mensagem ERP/1.0 é um único objeto JSON codificado em UTF-8, com os campos obrigatórios type e payload, seguido de uma quebra de linha como delimitador de quadro.

```
{"type": "ACTION", "payload": {"command": "examinar mesa"}}
```

Como o TCP entrega um fluxo contínuo de bytes, servidor e cliente mantêm buffers de leitura. Sempre que uma quebra de linha aparece, o conteúdo anterior é tratado como uma mensagem completa. Se a linha recebida não for JSON válido ou não contiver type/payload, o servidor responde ERROR [INVALID_ACTION] e descarta a linha.

6.2. Máquina de estados do servidor

Estado	Significado
WAITING_PLAYERS	Lobby. Jogadores podem entrar, votar no mapa e confirmar ready.
COUNTDOWN	Contagem regressiva antes da partida. Não permite novos jogadores.
IN_GAME	Partida em andamento. Jogadores enviam ACTION e CHAT; servidor controla cronômetro e estado das salas.
GAME_OVER	Partida encerrada por vitória ou derrota. Depois de 10 segundos, o servidor reseta para WAITING_PLAYERS.

- [*] -> WAITING_PLAYERS
- WAITING_PLAYERS -> COUNTDOWN: todos prontos e roles essenciais completos
- COUNTDOWN -> WAITING_PLAYERS: jogadores insuficientes ou role essencial ausente
- COUNTDOWN -> IN_GAME: contagem chegou a 0

- IN_GAME -> WAITING_PLAYERS: role essencial perdido por desconexão
- IN_GAME -> GAME_OVER: vitória, tempo esgotado ou menos de 2 jogadores
- GAME_OVER -> WAITING_PLAYERS: reset automático após 10 segundos

Máquina de estados do servidor — ERP/1.0

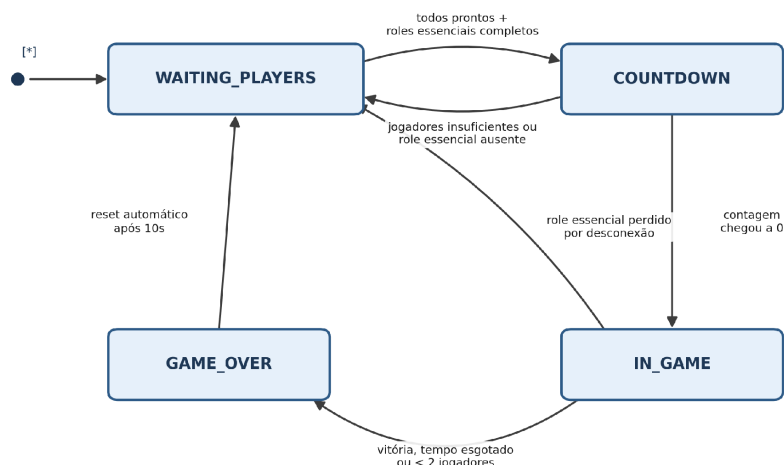


Figura 1 — Transições de estado do servidor ERP/1.0.

6.3. Mensagens Cliente -> Servidor

Mensagem	Payload	Estado válido	Efeito
JOIN	{"username": str}	WAITING_PLAYERS e antes do player_id	Tenta registrar jogador. Em sucesso, WELCOME. Em falha, ERROR e pode tentar JOIN novamente.
READY	{}	WAITING_PLAYERS	Marca jogador como pronto; se todos prontos e roles completos, inicia COUNTDOWN.
MAP_VOTE	{"map": str}	WAITING_PLAYERS	Registra ou substitui voto de mapa.
ACTION	{"command": str}	IN_GAME	Executa comando de jogo no GameState. Fora de IN_GAME gera NOT_IN_GAME.
CHAT	{"message": str}	Qualquer estado após JOIN	Redistribui mensagem textual a todos via CHAT_BROADCAST.
DISCONNECT	{}	Qualquer estado	Saída voluntária, tratada como desconexão.

Observação sobre READY

a confirmação de prontidão é unidirecional. Uma vez que o servidor recebe READY de um jogador, ele é marcado como pronto e não há, na versão atual do protocolo, uma mensagem para desfazer essa marcação (um “des-ready”). O estado de prontidão de todos os jogadores só é limpo automaticamente pelo servidor: ao entrar um novo jogador no lobby, ao reiniciar a partida ou ao cancelar a contagem regressiva. Um cliente mal-comportado não tem como reverter sua própria confirmação sem desconectar e reconectar.

Resolução da votação de mapa (MAP_VOTE)

ao final do COUNTDOWN, o servidor apura os votos recebidos via MAP_VOTE e seleciona o mapa com maior número de votos. Em caso de empate — incluindo o caso em que nenhum jogador votou — o servidor escolhe o primeiro mapa cadastrado internamente na lista de mapas disponíveis. Como a versão atual do projeto possui apenas o mapa hospital, essa regra de desempate nunca é exercitada na prática, mas já está implementada e pronta para quando novos mapas forem adicionados.

6.4. Mensagens Servidor -> Cliente

Mensagem	Tipo de envio	Quando é enviada
WELCOME	Unicast	Resposta direta a JOIN aceito.
MAP_VOTE_STATE	Unicast no JOIN; broadcast em MAP_VOTE	Informa votos atuais e mapas disponíveis.
MAP_SELECTED	Broadcast	Ao final do COUNTDOWN, quando o mapa vencedor é escolhido.
LOBBY_UPDATE	Broadcast	JOIN, READY, desconexão em lobby e reset automático.
COUNTDOWN	Broadcast	A cada segundo da contagem regressiva.
GAME_START	Unicast individual	Início do jogo; cada jogador recebe sua própria sala inicial.
ACTION_RESULT	Unicast	Resultado direto do comando enviado pelo jogador.

Mensagem	Tipo de envio	Quando é enviada
ROOM_UPDATE	Broadcast ou envio direcionado por sala	Quando muda o estado visível de uma sala: itens, portas, entrada/saída de jogadores ou item dropado.
PLAYER_EVENT	Broadcast	Eventos notáveis como joined, left, solved, moved, countdown_cancelled e match_reset.
CHAT_BROADCAST	Broadcast	Redistribuição de mensagem enviada via CHAT.
TIMER_UPDATE	Broadcast	A cada 120 segundos ou em marcos críticos de tempo.
HINT	Unicast	Resposta ao comando dica.
GAME_OVER	Broadcast	Vitória, derrota por tempo ou menos de 2 jogadores.
ERROR	Unicast	Rejeição de mensagem ou erro de protocolo/jogo.

6.4.1. Unicast, broadcast e atualizações direcionadas

O servidor escolhe o destinatário de cada mensagem conforme sua finalidade. Mensagens unicast são enviadas apenas ao cliente interessado; mensagens broadcast são enviadas a todas as conexões ativas; e algumas atualizações de sala podem ser enviadas somente aos jogadores que permanecem em uma sala específica.

- **Unicast:** usado quando a informação interessa apenas a um jogador, como WELCOME, GAME_START individual, ACTION_RESULT, HINT e ERROR.
- **Broadcast:** usado quando todos precisam saber de uma mudança global ou cooperativa, como LOBBY_UPDATE, COUNTDOWN, MAP_SELECTED, PLAYER_EVENT, CHAT_BROADCAST, TIMER_UPDATE e GAME_OVER.
- **ROOM_UPDATE híbrido:** em ações comuns, pode ser transmitido a todos e filtrado no cliente por players_here. Em casos sensíveis, como desconexão com item dropado, o servidor envia a atualização apenas aos jogadores que continuam naquela sala, já filtrando os objetos conforme o role de cada jogador.

Esse desenho evita tráfego desnecessário e também impede que um jogador veja objetos exclusivos de outro caminho quando uma sala é compartilhada.

6.4.2. Funcionamento do chat

O chat é implementado com duas mensagens: o cliente envia CHAT com o texto digitado e o servidor redistribui CHAT_BROADCAST a todos os jogadores conectados. O chat não depende do estado IN_GAME; basta o jogador já ter concluído o JOIN.

A função do chat é central para a jogabilidade. Como o jogo não anuncia automaticamente todo item encontrado, jogadores precisam compartilhar pistas, senhas e descobertas. Exemplo:

Cliente -> Servidor: CHAT {"message": "Achei a senha 1968"}

Servidor -> Todos: CHAT_BROADCAST {"from": "Ana", "message": "Achei a senha 1968"}

6.4.3. Temporização e marcos críticos de tempo

Cada partida possui um limite de 30 minutos (1800 segundos), contados a partir do envio do GAME_START. Esse valor é compartilhado por todos os jogadores e controlado inteiramente pelo servidor, em uma thread dedicada (_timer_loop) que verifica o tempo restante a cada segundo.

O servidor envia TIMER_UPDATE em duas situações:

- **Periodicamente**, a cada 120 segundos (2 minutos) de jogo;
- **Em marcos críticos**, quando o tempo restante atinge exatamente 300, 60, 30 ou 10 segundos — independentemente do intervalo periódico — para reforçar o senso de urgência perto do fim da partida.

Quando o tempo restante chega a zero, o servidor dispara automaticamente GAME_OVER com resultado de derrota ("lose"), encerrando a partida mesmo que os jogadores não tenham percebido o aviso anterior.

6.4.4. Exemplos de mensagens ERP/1.0

A seguir são apresentados exemplos simplificados de mensagens trocadas entre cliente e servidor durante uma partida.

Entrada de um jogador

Cliente → Servidor

```
{
  "type": "JOIN",
  "payload": {
    "username": "Lucas"
  }
}
```

Servidor → Cliente

```
{
  "type": "WELCOME",
  "payload": {
    "player_id": 2,
    "role": 0
  }
}
```

Execução de uma ação

Cliente → Servidor

```
{
  "type": "ACTION",
  "payload": {
    "command": "examinar terminal"
  }
}
```

Servidor → Cliente

```
{
  "type": "ACTION_RESULT",
  "payload": {
    "message": "O terminal exibe uma sequência numérica."
  }
}
```

Mensagem de chat

Cliente → Servidor

```
{
  "type": "CHAT",
  "payload": {
    "message": "Encontrei a senha 1968."
  }
}
```

Servidor → Todos

```
{
  "type": "CHAT_BROADCAST",
  "payload": {
    "from": "Lucas",
    "message": "Encontrei a senha 1968."
  }
}
```

Mensagem de erro

Servidor → Cliente

```
{
  "type": "ERROR",
```

```
"payload": {  
  "code": "NOT_IN_GAME",  
  "message": "A partida ainda não foi iniciada."  
}
```

6.5. Códigos de erro

Código	Quando é gerado
NAME_TAKEN	JOIN com nome de usuário já utilizado por outro jogador conectado.
GAME_FULL	JOIN quando a partida já atingiu 4 jogadores.
GAME_IN_PROGRESS	JOIN quando o servidor não está em WAITING_PLAYERS.
INVALID_ACTION	Nome vazio, mapa inválido, tipo desconhecido, JSON malformado ou comando antes do JOIN.
NOT_IN_GAME	ACTION enviada fora do estado IN_GAME.

6.6. Fluxo resumido de uma partida

Fluxo resumido de uma partida — ERP/1.0 (seção 6.6)

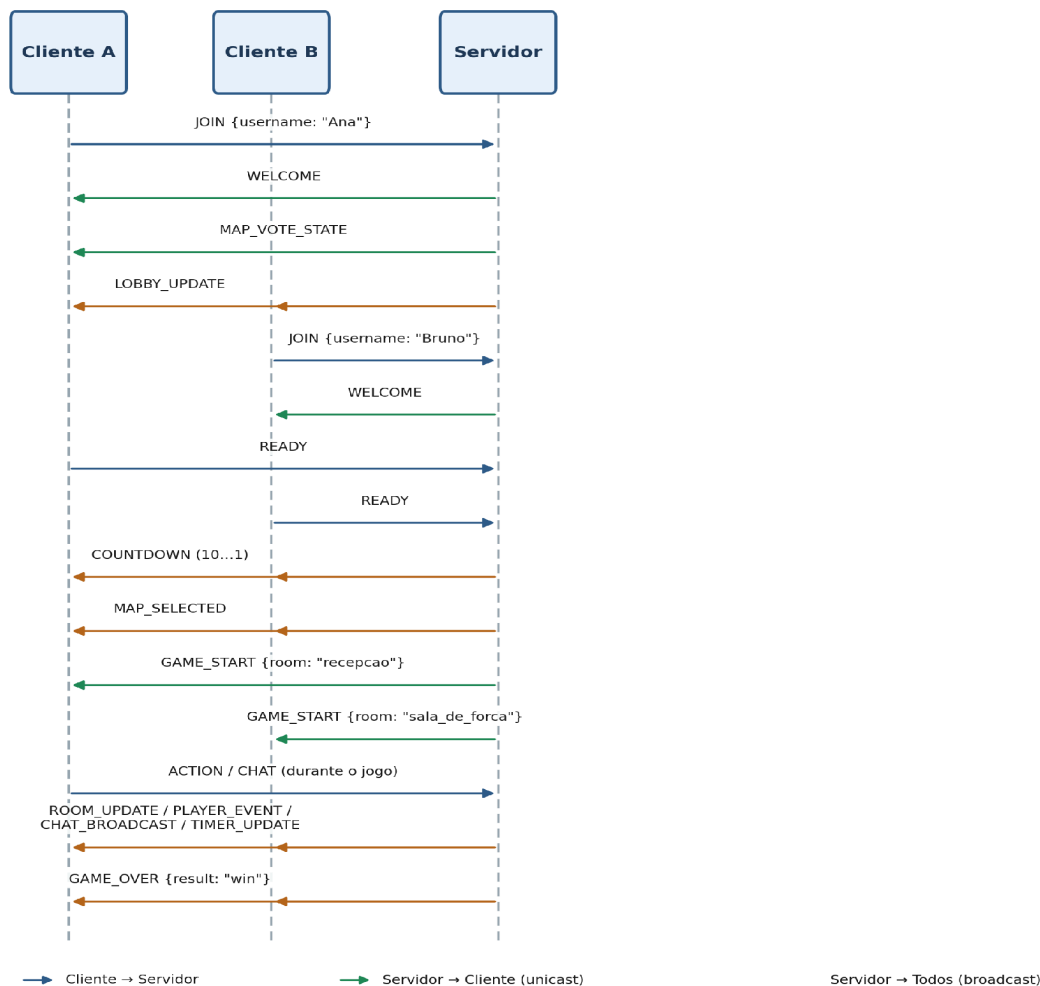


Figura 2 — Diagrama de sequência simplificado de uma partida completa.

6.7. Concorrência e consistência

Cada cliente é atendido por uma thread dedicada. Como todas compartilham o GameState e o dicionário de conexões, operações de JOIN, READY, ACTION, CHAT e desconexão são processadas dentro de um lock. Isso faz com que, do ponto de vista do estado do jogo, uma mensagem seja processada de cada vez, evitando corrupção de estado em ações concorrentes.

7. Subprotocolo de descoberta de servidor (UDP broadcast)

O `launcher.py` implementa um subprotocolo auxiliar sobre UDP para que um jogador consiga encontrar automaticamente uma partida já criada na rede local. Esse mecanismo não substitui o ERP/1.0: ele só descobre onde está o servidor TCP.

Quando alguém escolhe Criar partida, o launcher inicia o servidor TCP e, paralelamente, transmite periodicamente uma mensagem UDP broadcast na porta 5001. A mensagem tem formato de texto simples:

```
ESCAPE_ROOM_ERP1:<ip_do_servidor>:<porta_tcp_do_jogo>
```

Quando outro jogador escolhe Entrar em partida, o launcher abre um socket UDP na porta 5001 e escuta por até alguns segundos. Se receber uma mensagem com o prefixo `ESCAPE_ROOM_ERP1:`, extrai o IP e a porta TCP e então abre a conexão real do jogo via TCP. Se nada for encontrado, o cliente entra no modo manual e pede o IP do servidor por teclado.

O uso de UDP é adequado aqui porque a descoberta é tolerante a perdas. Se um pacote de anúncio for perdido, outro será enviado pouco depois. Já o jogo em si permanece em TCP para garantir confiabilidade, ordem e conexão persistente.

8. Limitações conhecidas

- O projeto possui apenas um mapa jogável, hospital, embora a infraestrutura de votação já esteja preparada para múltiplos mapas no futuro.
- A descoberta por UDP broadcast só funciona dentro do mesmo segmento de rede local. Em redes com isolamento de clientes, VPNs ou múltiplas sub-redes, o modo manual por IP deve ser usado.
- No Windows nativo, o cliente pode exigir `pyreadline3` ou execução via WSL por causa do uso de `readline` para preservar melhor o input do usuário no terminal.
- O servidor documenta e tolera a repetição de `JOIN` antes do `WELCOME` para o fluxo de nome já usado. Um cliente mal-comportado que tente fugir do cliente de referência pode exigir validações extras em versões futuras.

9. Instruções de execução

As dependências completas já estão descritas na seção 4 (Requisitos mínimos de funcionamento); aqui o foco é apenas nos comandos de execução.

9.1. Forma recomendada: `launcher.py`

A maneira mais simples de rodar o jogo, especialmente em apresentação, é através do launcher, que já cuida da descoberta automática de servidor via UDP broadcast (seção 7).

No computador que será o host:


```
python3 launcher.py
```

Em seguida, escolher a opção:

1. Criar partida

Esse computador inicia o servidor TCP, entra como jogador local e passa a transmitir periodicamente sua presença na rede via UDP broadcast.

Nos computadores dos demais jogadores:

```
python3 launcher.py
```

Em seguida, escolher a opção:

2. Entrar em partida

O launcher tenta localizar automaticamente o servidor na rede local por até alguns segundos. Se encontrar, conecta sozinho; se não encontrar, solicita o IP do servidor manualmente (Enter conecta em 127.0.0.1).

9.2. Forma manual: server.py e client.py

Também é possível rodar servidor e clientes separadamente, sem o launcher — útil para depuração ou quando o broadcast UDP não está disponível (por exemplo, redes com isolamento de clientes).

No computador servidor:

```
python3 server.py
```

Por padrão o servidor escuta em 0.0.0.0 na porta TCP 5000. Host e porta podem ser sobrescritos:

```
python3 server.py --host 0.0.0.0 --port 5000
```

Nos computadores clientes:

```
python3 client.py --host IP_DO_SERVIDOR
```

Exemplo em rede local:

```
python3 client.py --host 192.168.0.10
```

Para testar tudo na mesma máquina (sem outros computadores na rede):

```
python3 client.py --host 127.0.0.1
```

9.3. Gerando um executável standalone (opcional)

Para distribuir o cliente sem exigir Python instalado na máquina do jogador, é possível empacotar o launcher (ou o client.py) com o PyInstaller:

```
python -m pip install pyinstaller
```

```
pyinstaller --onefile launcher.py
```

O executável gerado fica disponível na pasta dist/. Essa etapa é opcional e não substitui os requisitos da seção 4 no computador que efetivamente roda o servidor.